

Dossier de soutenance

RNCP37827BC01

Réaliser la collecte, le stockage et la mise à disposition des données d'un projet en intelligence artificielle

PPMT

Plateforme Prédictive des Métiers en Tension — Île-de-France

Candidate	Claire Lucie DIOUF
Formation	Développeur en Intelligence Artificielle — Artefact
Projet	Pipeline de données et API sur les offres d'emploi franciliennes (France Travail + Adzuna)
Binôme	Bernard GBOHOUGNON
Dépôt	github.com/Clairelucie/PPMT
Soutenance	15 octobre 2026

Sommaire

1. Présentation du service numérique.....	3
2. Cheminement du projet : de la V1 à la V2.....	4
3. Architecture générale du pipeline.....	5
4. C1 — Collecter les données de manière automatisée.....	6
5. C2 — Préparer les données pour leur première utilisation.....	8
6. C3 — Mettre en œuvre des règles d'agrégation de données.....	11
7. C4 — Créer une base de données.....	14
8. C5 — Mettre à disposition les données via une API.....	17
9. Tests et qualité.....	21
10. Service numérique et ouverture Machine Learning.....	22
11. Bilan et perspectives.....	24
Annexes.....	26

1. Présentation du service numérique

Contexte et problématique

Selon l'enquête Besoins en Main-d'Œuvre (BMO) 2025 de France Travail, la moitié des projets de recrutement sont jugés difficiles par les employeurs. Les informations sur les métiers qui recrutent sont pourtant dispersées : chaque site d'emploi publie ses propres offres, avec ses propres catégories, et les statistiques officielles paraissent une fois par an.

PPMT (Plateforme Prédictive des Métiers en Tension) rassemble chaque jour les offres d'emploi publiées en Île-de-France par **France Travail** et par l'agrégateur **Adzuna**, les harmonise sur le référentiel des métiers **ROME**, et calcule pour chaque métier un **indice de tension** qui mesure la pression de recrutement exercée par les employeurs.

Problématique

Comment collecter, fiabiliser et mettre à disposition, de façon automatisée, les offres d'emploi franciliennes de plusieurs sources afin d'identifier les métiers qui recrutent le plus et d'orienter candidats, recruteurs et organismes de formation ?

Ce que fait le service

- **Collecte** quotidienne des offres via deux API (France Travail, Adzuna) et deux fichiers de référence géographiques : les contours des départements (carte, densité) et la liste des communes d'Île-de-France (pour retrouver le département de chaque offre).
- **Nettoyage et harmonisation** : dédoublonnage, salaires en € bruts annuels (les salaires non affichés sont complétés par une médiane), contrats normalisés, départements, masquage des données personnelles.
- **Stockage** dans une base relationnelle SQLite de 4 tables, avec contraintes d'intégrité et indicateurs agrégés en SQL.
- **Mise à disposition** par une **API REST FastAPI** sécurisée (7 endpoints, documentation Swagger) et par un **dashboard Streamlit** de 6 onglets.

Utilisateurs cibles

Utilisateur	Usage de PPMT
Cabinets de recrutement	Repérer les métiers où la concurrence entre employeurs est forte ; ajuster délais et honoraires
Candidats et conseillers emploi	Savoir quels métiers recrutent réellement, dans quel département, avec quel type de contrat
Organismes de formation	Aligner l'offre de formation (CPF, OPCO) sur les métiers les plus demandés
Développeurs / data analysts	Interroger les données via l'API pour les intégrer à leurs propres outils

Mon rôle dans le projet

Le projet a été mené **en binôme avec Bernard GBOHOUGNON**, chacun avec un rôle défini. En V1 (mai 2026), j'ai pris en charge la collecte France Travail (authentification OAuth2), le dashboard Streamlit et la partie Machine Learning ; Bernard a collecté et nettoyé les données Adzuna et administré le dépôt Git. Pour la **refonte V2** présentée dans ce dossier, nous avons de nouveau réparti le travail : j'ai réécrit la collecte, le nettoyage et le stockage en modules testables, créé l'API et les tests automatisés, et corrigé la partie ML ; Bernard a mis en place la journalisation (logs) et l'idempotence du pipeline.

2. Cheminement du projet : de la V1 à la V2

En relisant les critères du bloc RNCP37827BC01, j'ai constaté que la V1 mettait l'accent sur la visualisation, alors que le bloc évalue la chaîne complète **collecte → préparation → stockage → mise à disposition par API**.

Plusieurs écarts ont été identifiés et corrigés :

Dimension	V1 (mai 2026)	V2 (octobre 2026)
Collecte	Script France Travail de test (10 offres, Paris) ; collecte Adzuna hors dépôt	collect.py : 2 API + 2 fichiers de référence, pagination, reprise sur erreur, logs, idempotence
Préparation	Scripts séparés, chiffres de rapport codés en dur	prepare.py : règles unitaires testées, schéma commun aux deux sources
Stockage	1 table remplie sur 3, sans clé étrangère	4 tables normalisées, clés, contraintes CHECK/UNIQUE, 6 index
Agrégation	Calcul pandas	Requête SQL (GROUP BY + fonction fenêtre)
Mise à disposition	Dashboard Streamlit uniquement	API REST FastAPI sécurisée + Swagger + Docker, dashboard conservé
RGPD	Non traité	Pseudonymisation, purge 12 mois, registre de traitement
Tests	Aucun	57 tests pytest (collecte simulée, règles, base, API)
Machine Learning	$R^2 = 0,995$ annoncé	Fuite de données identifiée et corrigée (voir §10)

Cette démarche illustre une compétence que je considère centrale : reconnaître l'écart entre ce qui a été produit et ce qui est attendu, puis corriger, y compris lorsqu'un résultat flatteur ($R^2 = 0,995$) se révèle être un artefact.

3. Architecture générale du pipeline

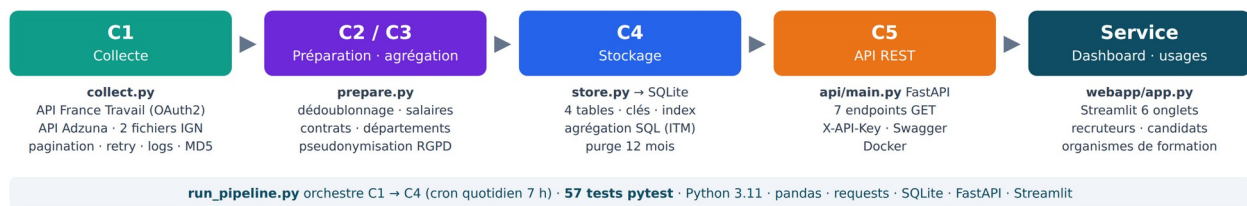


Figure 1 — Chaîne de traitement PPMT, des sources au service

Chaque compétence du bloc correspond à un module Python indépendant, testable et exécutable seul.

`run_pipeline.py` enchaîne les étapes C1 → C4 ; il est planifié chaque jour à 7 h par cron. L'API (C5) lit la base en lecture seule.

Arborescence du dépôt

```

PPMT/
├── src/
│   ├── collect.py      C1 – API France Travail, API Adzuna, contours et communes IDF
│   ├── prepare.py      C2/C3 – nettoyage, normalisation, dédoublonnage, RGPD
│   └── store.py         C4 – schéma SQLite, chargement, agrégation SQL, purge
├── api/main.py          C5 – API REST FastAPI
├── tests/test_pipeline.py 57 tests pytest
├── run_pipeline.py       orchestrateur C1 → C4
├── webapp/app.py         dashboard Streamlit
├── notebook/ml_tests.py  expérimentation ML
├── docs/ dictionnaire.md · rgpd.md
└── Dockerfile            image de l'API

```

Couche	Technologies
Collecte	Python 3.11 · requests · python-dotenv · OAuth2 client_credentials
Préparation	pandas · expressions régulières · hashlib
Stockage	SQLite 3 (clés étrangères, CHECK, fonctions fenêtre)
API	FastAPI · Pydantic · Uvicorn · Docker
Tests	pytest · FastAPI TestClient (httpx)
Visualisation	Streamlit · Plotly

4. C1 — Collecter les données de manière automatisée

Sources de données

Source	Type	Accès	Contenu	Volume
France Travail — Offres d'emploi v2	API REST JSON	OAuth2 (client_id / secret)	Offres, code ROME natif, contrat, salaire, expérience	24 051 offres (8 dépts)
Adzuna	API REST JSON	app_id / app_key	Offres agrégées de nombreux sites, catégorie, salaire	12 500 brutes → 4 626
GeoJSON départements (IGN / Insee)	Fichier	HTTPS public	Contours des 8 départements : centre et superficie en base, carte	8 entités
Communes d'Île-de-France (IGN / Insee)	Fichier	HTTPS public	Code Insee, nom et département : localisation des offres Adzuna	1 276 communes

Les identifiants sont lus dans un fichier `.env` exclu du dépôt (`.gitignore`) ; un modèle `.env.example` documente les variables attendues.

Authentification OAuth2 et pagination France Travail

L'API France Travail renvoie au maximum 150 offres par appel et refuse une plage qui commence au-delà de 3 000. Pour couvrir l'Île-de-France, `collect.py` interroge chaque département séparément et parcourt toutes les pages (0-149, 150-299 ... 3000-3149). C'est ce qui explique la répartition très homogène des offres France Travail (environ 3 000 par département).

src/collect.py – obtention du jeton

```
def ft_token(session: requests.Session, client_id: str, client_secret: str) -> str:
    r = session.post(FT_TOKEN_URL, data={
        "grant_type": "client_credentials",
        "client_id": client_id,
        "client_secret": client_secret,
        "scope": "api_offresdemploiv2 o2dsoffre",
    }, timeout=30)
    r.raise_for_status()
    token = r.json().get("access_token")
    if not token:
        raise RuntimeError("Token France Travail absent de la réponse")
    return token
```

src/collect.py – pagination complète par département

```
def ft_offres_departement(session: requests.Session, token: str, dept: str) -> list[dict]:
    """Parcourt toutes les pages d'un département : range=0-149, 150-299, ... jusqu'à 3000-3149."""
    headers = {"Authorization": f"Bearer {token}", "Accept": "application/json"}
    offres: list[dict] = []
    for start in range(0, FT_MAX_START + 1, FT_PAGE):
        params = {"departement": dept, "range": f"{start}-{start + FT_PAGE - 1}"}
        r = get_with_retry(session, FT_SEARCH_URL, params=params, headers=headers)
        if r.status_code == 204:
            # plus aucun résultat
            break
        page = r.json().get("resultats", [])
        offres.extend(page)
        if len(page) < FT_PAGE:
            # dernière page atteinte
```

```

        break
        time.sleep(0.1)                # quota : 10 appels / seconde
    log.info(f" France Travail dépt {dept} : {len(offres)} offres")
    return offres

```

Robustesse de la collecte

src/collect.py – reprise sur erreur

```

def get_with_retry(session: requests.Session, url: str, *, params=None, headers=None,
                  tries: int = 3, timeout: int = 30) -> requests.Response:
    """GET avec 3 tentatives et back-off exponentiel (1 s, 2 s, 4 s).
    Respecte l'en-tête Retry-After renvoyé avec un HTTP 429."""
    for attempt in range(1, tries + 1):
        try:
            r = session.get(url, params=params, headers=headers, timeout=timeout)
            if r.status_code == 429:
                wait = int(r.headers.get("Retry-After", 2 ** attempt))
                log.warning(f"HTTP 429 – pause {wait}s (tentative {attempt}/{tries})")
                time.sleep(wait)
                continue
            r.raise_for_status()
            return r
        except requests.RequestException as e:
            if attempt == tries:
                raise
            log.warning(f"Erreur réseau ({e}) – nouvelle tentative dans {2 ** (attempt - 1)}s")
            time.sleep(2 ** (attempt - 1))
    raise RuntimeError(f"Échec après {tries} tentatives : {url}")

```

Mécanisme	Mise en œuvre
Reprise sur erreur	3 tentatives, attente exponentielle (1 s, 2 s, 4 s) ; respect de l'en-tête Retry-After d'un HTTP 429
Respect des quotas	Pause de 100 ms entre deux pages France Travail (limite : 10 appels/s)
Idempotence	Un fichier brut par source et par jour ; déjà présent → non re-téléchargé (--force pour forcer)
Traçabilité	Journal horodaté logs/collect_AAAAMMJJ_HHMMSS.log (fichier + console) ; empreinte MD5 de chaque fichier
Isolation des échecs	Une source en échec n'arrête pas les autres ; code retour 1 si une source échoue (exploitable par cron / CI)
Contrôle des fichiers	GeoJSON : les 8 départements doivent être présents ; communes : au moins 1 200, réparties sur les 8 départements, sinon exception
Aplatissement	ft_aplatir() / adzuna_aplatir() transforment le JSON imbriqué en lignes plates au schéma attendu par C2

Exécution réelle – journal de collecte

```

$ python src/collect.py --source geojson
15:50:45 | INFO | === PPMT – Collecte C1 ===
15:50:45 | INFO | GeoJSON : 8 départements IDF → departements_idf.geojson (MD5
ec919e5ad96f6219a8d8d0e5b6ad5da7)
15:50:45 | INFO | Terminé – 1/1 sources OK – journal : collect_20260928_155045.log
$ python src/collect.py --source geojson                # 2e exécution : idempotence
15:50:46 | INFO | GeoJSON : déjà présent (MD5 ec919e5ad96f6219a8d8d0e5b6ad5da7) – ignoré

```

5. C2 — Préparer les données pour leur première utilisation

Les deux sources n'ont ni le même schéma, ni les mêmes conventions. `prepare.py` applique des **règles unitaires** (une fonction = une règle, chacune testée) puis projette les deux sources sur un **schéma commun de 19 colonnes**.

Problèmes détectés et méthodes retenues

Pour chaque problème, j'ai comparé la méthode de la V1 et celle de la V2, et retenu la combinaison qui garde le plus de données tout en restant fiable.

Problème	Méthode retenue	Justification	Résultat
Salaires non affichés : 64 % des offres (63 % Adzuna, 65 % France Travail)	Médiane des salaires affichés du groupe le plus précis (≥ 3 salaires) : métier $\times$ département $\rightarrow$ métier $\rightarrow$ domaine ROME $\rightarrow$ département $\rightarrow$ Île-de-France. Colonnes <code>salaire_impute</code> et <code>salaire_source</code>	Médiane robuste aux salaires extrêmes (V1). Le métier explique davantage le salaire que le département (V2). Les indicateurs n'utilisent que les salaires affichés : pas d'enrichissement circulaire	100 % des offres ont un salaire. Affichés : 10 270 ; métier $\times$ département : 9 268 ; métier : 5 266 ; domaine : 2 554 ; département : 845 ; région : 16
Doublons : même offre publiée plusieurs fois ou sur les deux sources	URL, puis empreinte titre + entreprise + lieu, entre les deux sources ; priorité à France Travail	Une même offre a des URL différentes (V1) et peut être publiée sur les deux sources (V2)	Adzuna 12 500 $\rightarrow$ 4 626 (V1), puis 361 doublons de plus ; 0 doublon en base (UNIQUE)
Entreprises non renseignées (64,7 % France Travail)	« Non renseigné », exclu du comptage des entreprises	Ne rien inventer ; garder <code>nb_entreprises</code> juste	0 valeur vide
Salaires horaires, mensuels, annuels ; bornes inversées	$\times 1\,820$ h ou $\times 12$; bornes 10 k€ – 300 k€ ; min et max permutés si inversés	Comparer des montants de même nature	149 fourchettes corrigées
Contrats hétérogènes (« CDD - 12 Mois », « permanent », « Temps plein »)	7 catégories : CDI, CDD, Intérim, Alternance, Libéral, Autre, Non renseigné ; temps de travail Adzuna dans une colonne à part	Plus précis que les 4 catégories de la V1, sans confondre « non renseigné » et « autre »	Croisement des deux sources possible
Dates en formats mixtes	ISO 8601 en base ; vue <code>v_offres_analyse</code> : année, mois, âge, offre récente	Colonnes d'analyse de la V1 conservées, calculées à la volée (pas de redondance)	Tri, filtres et âge des offres
Codes ROME absents chez Adzuna	1. intitulé identique à des offres France Travail ; 2. catégorie Adzuna $\rightarrow$ ROME (V1) ; 3. vide. Code fourre-tout M1607 écarté	L'intitulé est plus précis que la catégorie : Adzuna classe « Comptable » en restauration	596 codes par intitulé, 2 763 par catégorie ; 1 169 offres sans code, exclues du calcul de tension
Lieux Adzuna sans département (« Montévrain, Lagny-sur-Marne »)	Code du libellé, nom de département, puis référentiel des 1 276 communes d'Île-de-France (IGN / Insee) et pôles connus (La Défense)	Remplace les listes de villes écrites à la main, qui se trompaient (Viry-Châtillon classé dans le 92)	Sans département : 675 $\rightarrow$ 81 (lieu « Île-de-France » seul) ; 47 départements corrigés ; 15 offres hors région

Problème	Méthode retenue	Justification	Résultat
			supprimées
Intitulés bruités	Espaces normalisés ; mention « (H/F) » retirée	Regrouper les mêmes intitulés	1 823 intitulés nettoyés
Coordonnées de recruteurs dans les descriptions	Masquage [email] / [tel]	RGPD : minimisation	4 e-mails et 5 téléphones masqués

Exemples de règles

src/prepare.py

```
def normaliser_salaire_annuel(val):
    """Ramène un montant en € brut annuel : < 100 → horaire (×1 820 h), < 5 000 → mensuel (×12).
    Hors bornes [10 000 ; 300 000] → valeur aberrante, remplacée par NaN."""
    if val is None or pd.isna(val):
        return None
    val = float(val)
    if val < 100:
        val *= 1820
    elif val < 5000:
        val *= 12
    return val if 10_000 <= val <= 300_000 else None
```

src/prepare.py – salaires non affichés : cascade de médianes

```
def imputer_salaires(df: pd.DataFrame, min_obs: int = 3) -> tuple[pd.DataFrame, dict]:
    """Salaire non affiché → médiane des salaires AFFICHÉS du groupe le plus précis qui en compte
    au moins `min_obs` :
    1. même métier ROME dans le même département (ex. aide-soignant dans le 93)
    2. même métier ROME en Île-de-France
    3. même domaine ROME (3 premiers caractères, ex. J15 = soins)
    4. même département
    5. toute l'Île-de-France
    Combine la V1 (médiane par département puis par catégorie) et la V2 (médiane par métier).
    La médiane, pas la moyenne : quelques salaires très élevés ne la déplacent pas.
    salaire_min / salaire_max restent vides (on n'invente pas de fourchette) ; salaire_impute = 1
    et
    salaire_source indiquent la valeur calculée, et les indicateurs n'utilisent que les salaires
    affichés :
    c'est ce qui évite l'« enrichissement circulaire » (des valeurs imputées qui se confirmeraient
    elles-mêmes)."""
    df = df.copy()
    for col in ("code_rome", "code_dept"):
        if col not in df:
            df[col] = None
    affiche = df["salaire_moyen"].notna()
    df["salaire_impute"] = (~affiche).astype(int)
    df["salaire_source"] = pd.Series("affiché", index=df.index).where(affiche, None)
    df["_domaine"] = df["code_rome"].str[:3]
    obs = df[affiche]
    niveaux = [(["code_rome", "code_dept"], NIVEAUX_SALAIRE[1]), (["code_rome"],
    NIVEAUX_SALAIRE[2]),
    (["_domaine"], NIVEAUX_SALAIRE[3]), (["code_dept"], NIVEAUX_SALAIRE[4])]
    for cles, nom in niveaux:
        med = obs.groupby(cles)["salaire_moyen"].agg(["median", "count"])
        med = med.loc[med["count"] >= min_obs, "median"]
        a_remplir = df["salaire_moyen"].isna() & df[cles].notna().all(axis=1)
        if not a_remplir.any() or med.empty:
            continue
```

```

        idx = pd.MultiIndex.from_frame(df.loc[a_remplir, cles]) if len(cles) > 1 else
df.loc[a_remplir, cles[0]]
        valeurs = pd.Series(med.reindex(idx).to_numpy(), index=df.index[a_remplir])
        ok = valeurs.notna()
        df.loc[valeurs.index[ok], "salaire_moyen"] = valeurs[ok].round(0)
        df.loc[valeurs.index[ok], "salaire_source"] = nom
        mediane_globale = float(obs["salaire_moyen"].median())
        reste = df["salaire_moyen"].isna()
        df.loc[reste, "salaire_moyen"] = round(mediane_globale)
        df.loc[reste, "salaire_source"] = NIVEAUX_SALAIRE[5]
        stats = df["salaire_source"].value_counts().reindex(NIVEAUX_SALAIRE, fill_value=0).to_dict()
        stats["valeur_mediane_globale"] = round(mediane_globale)
        return df.drop(columns="_domaine"), stats

```

Pourquoi la médiane : sur les salaires affichés en Île-de-France, la moyenne est de 33 463 € et la médiane de 27 120 €. Quelques salaires très élevés tirent la moyenne vers le haut ; la médiane représente mieux une offre typique. Les indicateurs par métier et par département sont ensuite calculés sur les seuls salaires affichés, pour que les valeurs complétées ne se confirment pas elles-mêmes.

src/prepare.py – RGPD

```

def pseudonymiser(texte) -> str:
    """RGPD : masque les e-mails et numéros de téléphone de recruteurs présents dans le texte libre."""
    if texte is None or pd.isna(texte):
        return ""
    return RE_TEL.sub("[tel]", RE_EMAIL.sub("[email]", str(texte)))

```

src/prepare.py – dédoublonnage inter-sources

```

def dedoubler(df: pd.DataFrame) -> tuple[pd.DataFrame, dict]:
    """1) même URL 2) même empreinte. Priorité à France Travail (codes ROME natifs)."""
    n0 = len(df)
    df = df.sort_values("source", key=lambda s: s.ne("france_travail")) # FT en premier
    df = df[df["url"].isna() | ~df.duplicated("url")]
    n1 = len(df)
    df = df.drop_duplicates("hash_offre", keep="first")
    return df.reset_index(drop=True), {"doublons_url": n0 - n1, "doublons empreinte": n1 - len(df)}

```

Résultat

Sortie : [data/processed/offres_unifiees.csv](#) — **28 301 offres** (23 691 France Travail, 4 610 Adzuna), puis 28 219 en base après application de la fenêtre de conservation de 12 mois (§7).

6. C3 — Mettre en œuvre des règles d'agrégation de données

Les indicateurs par métier sont calculés **en SQL, directement dans la base**, après chaque chargement ; seul le salaire médian est calculé en Python (`store.calculer_medianes`), SQLite n'ayant pas de fonction médiane. Le code ROME sert de clé commune aux deux sources : natif chez France Travail, obtenu par correspondance catégorie → ROME chez Adzuna.

Règles d'agrégation

Indicateur	Règle	Justification
nb_offres_ft / adzuna / total	COUNT par source et par code ROME	Volume de la demande des employeurs
nb_entreprises	COUNT DISTINCT, « Non renseigné » exclu	Une offre massive d'un seul employeur ≠ tension de marché
nb_departements	COUNT DISTINCT code_dept	Diffusion géographique du besoin
part_cdi	Moyenne de (contrat = CDI) × 100	Qualité des emplois proposés
salaire_median	Médiane des salaires affichés (valeurs complétées exclues), calculée en Python car SQLite n'a pas de fonction MEDIAN	Attractivité du métier, sans être faussée par les salaires extrêmes
part_salaire_affiche	Part des offres qui affichent un salaire	Fiabilité du salaire médian ; transparence des employeurs
indice_tension	nb_offres_total ÷ moyenne des métiers × 100	100 = métier moyen ; comparable d'une collecte à l'autre
statut	Seuils 50 / 100 / 150	Lecture immédiate pour un public non technique

src/store.py – agrégation SQL (fonction fenêtre AVG() OVER ())

```
SQL_INDICATEURS = """
INSERT INTO indicateurs_tension
WITH agg AS (
    SELECT code_rome,
           SUM(source = 'france_travail') AS nb_offres_ft,
           SUM(source = 'adzuna') AS nb_offres_adzuna,
           COUNT(*) AS nb_offres_total,
           COUNT(DISTINCT NULLIF(entreprise, 'Non renseigné')) AS nb_entreprises,
           COUNT(DISTINCT code_dept) AS nb_departements,
           ROUND(AVG(contrat = 'CDI') * 100, 1) AS part_cdi,
           NULL AS salaire_median, -- calculée
ensuite (médiane)
           ROUND(AVG(salaire_impute = 0) * 100, 1) AS part_salaire_affiche
    FROM offres
    WHERE code_rome IS NOT NULL
    GROUP BY code_rome
)
SELECT code_rome, nb_offres_ft, nb_offres_adzuna, nb_offres_total, nb_entreprises,
       nb_departements, part_cdi, salaire_median, part_salaire_affiche,
       ROUND(nb_offres_total * 100.0 / AVG(nb_offres_total) OVER (), 1) AS indice_tension,
       CASE
           WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 150 THEN 'TRES EN
TENSION'
           WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 100 THEN 'EN TENSION'
```

```

        WHEN nb_offres_total * 100.0 / AVG(nb_offres_total) OVER () > 50 THEN 'EQUILIBRE'
        ELSE 'SATURE'
    END AS statut,
    :date_calcul
FROM agg;

```

Ce que mesure — et ne mesure pas — l'indice de tension

L'indice mesure la **pression de la demande des employeurs** : un métier à 1 850 publie 18,5 fois plus d'offres qu'un métier moyen. Il ne s'agit pas du ratio officiel offres / demandeurs d'emploi de la DARES : PPMT ne dispose pas du nombre de candidats par métier. C'est la principale piste d'enrichissement (§11).

Résultats

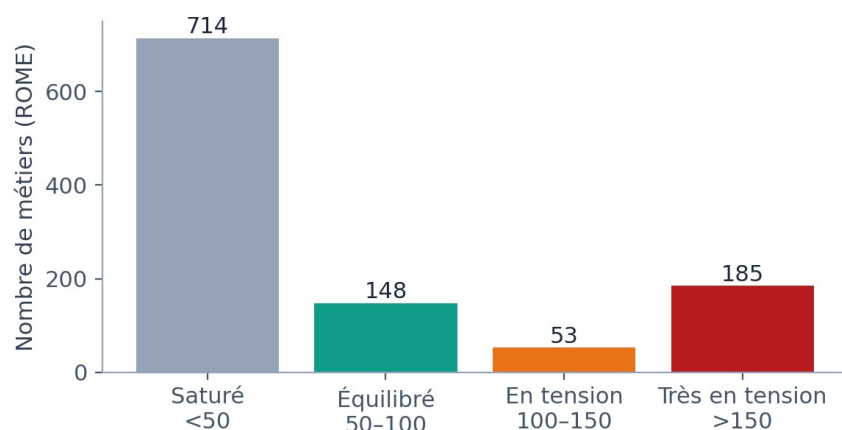


Figure 2 — Répartition des 1 100 métiers ROME par statut

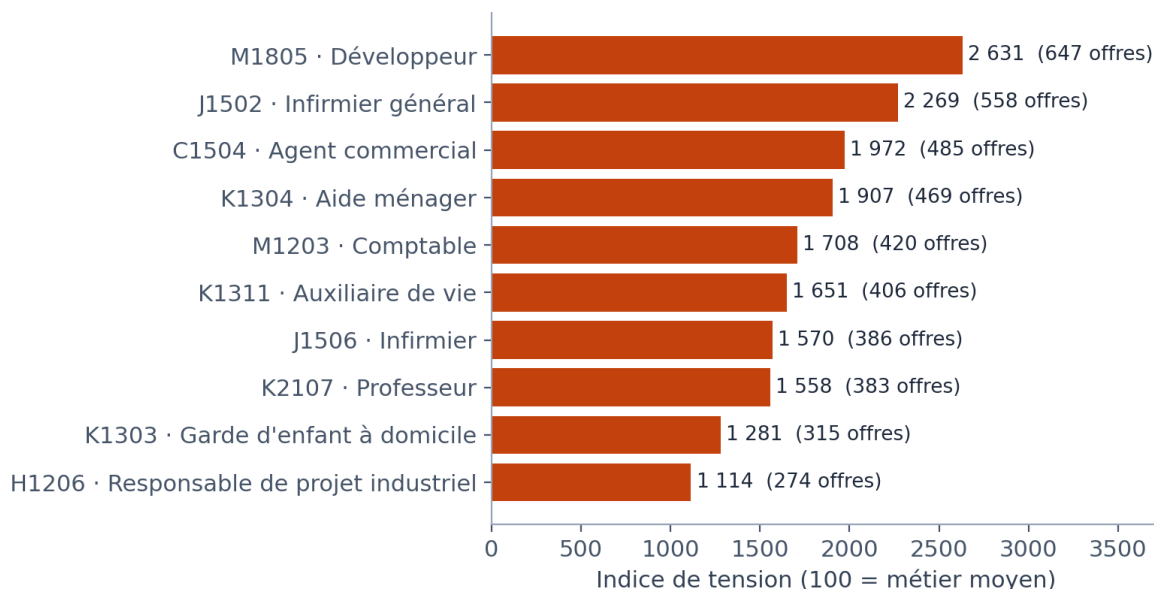


Figure 3 — Les 10 métiers à plus forte tension en Île-de-France (collecte mai 2026)

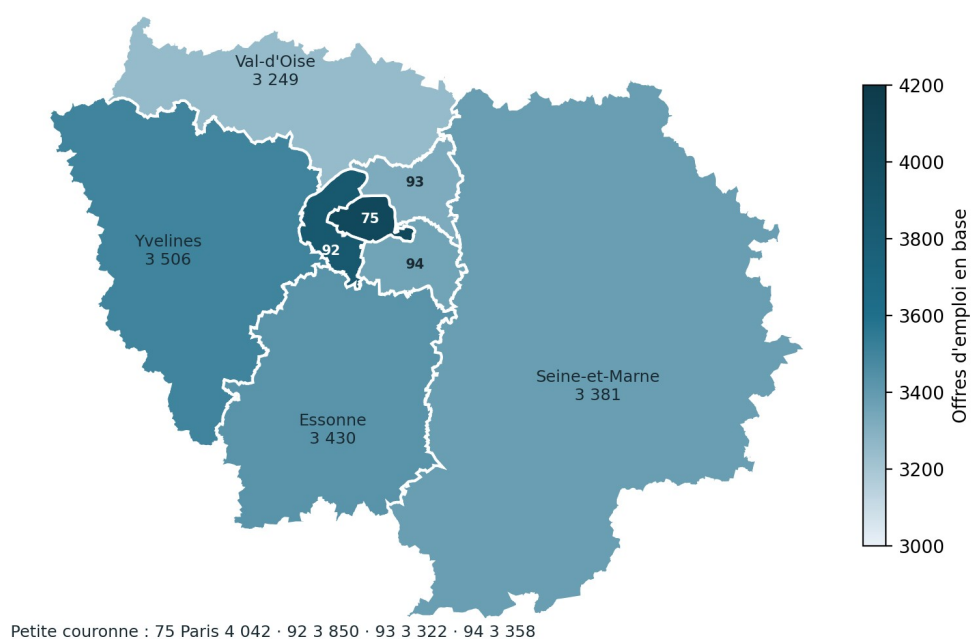


Figure 4 — Offres par département ; contours issus du fichier GeoJSON collecté en C1

7. C4 — Créer une base de données

Choix technologique

Critère	SQLite (retenu)	PostgreSQL	MongoDB (NoSQL)
Nature des données	Tabulaires, schéma stable	Idem	Adapté au JSON hétérogène
Volume (≈ 30 000 offres/jour)	Largement suffisant	Surdimensionné	Surdimensionné
Intégrité (clés, contraintes)	FK, CHECK, UNIQUE	Complet	Limitée
Agrégations	SQL + fonctions fenêtre	Idem	Pipeline d'agrégation
Déploiement	Un fichier, zéro serveur, embarqué dans l'image Docker	Serveur à administrer	Serveur à administrer

Les offres, une fois aplaties, ont une structure fixe et sont interrogées par filtres et agrégations : un modèle **relationnel** est le plus adapté. SQLite offre ces garanties sans serveur ; la migration vers PostgreSQL est prévue si l'écriture devient concurrente (plusieurs régions collectées en parallèle).

Modèle physique de données

departements · 8 lignes 📌 code_dept TEXT (2) CHECK nom TEXT NOT NULL latitude · longitude REAL CHECK (IDF) superficie_km2 REAL	offres · 28 177 lignes 📌 id INTEGER AUTOINC source TEXT CHECK (ft adzuna) id_source TEXT 📌 hash_offre TEXT UNIQUE titre TEXT NOT NULL entreprise · lieu TEXT 📌 code_dept → departements 📌 code_rome → metiers_rome categorie · experience TEXT contrat TEXT CHECK (7 valeurs) temps_travail TEXT salaire_min · salaire_max REAL CHECK 10k-300k salaire_moyen REAL (médiane si absent) salaire_impute · salaire_source INTEGER (0 1) · TEXT rome_source TEXT CHECK (3 valeurs) date_publication TEXT ISO 8601 description TEXT pseudonymisé url · date_import TEXT	indicateurs_tension · 1 100 📌 code_rome → metiers_rome nb_offres_ft INTEGER nb_offres_adzuna INTEGER nb_offres_total INTEGER nb_entreprises INTEGER nb_departements INTEGER part_cdi REAL (%) salaire_median REAL part_salaire_affiche REAL (%) indice_tension REAL statut TEXT date_calcul TEXT
--	--	--

📌 **clé primaire** · 📌 **clé étrangère** · 📌 **contrainte d'unicité (dédoublonnage)** · Relations : departements 1—N offres · metiers_rome 1—N offres · metiers_rome 1—1 indicateurs_tension (alimentée par agrégation SQL de offres) · Index : code_dept, code_rome, source, contrat, date_publication, statut · Vue *v_offres_analyse* : année, mois, âge, offre récente

Figure 5 — Modèle physique : 2 référentiels, 1 table de faits, 1 table d'agrégats

Le modèle respecte la 3e forme normale : les libellés de départements et de métiers ne sont stockés qu'une fois, dans leurs référentiels. **indicateurs_tension** est une table dérivée, recalculée à chaque exécution à partir de **offres**. La table **departements** reçoit aussi, depuis le fichier GeoJSON, le centre géographique (latitude, longitude) et la superficie de chaque département, calculés dans **store.py** : l'API s'en sert pour la densité d'offres, le dashboard pour la carte.

src/store.py — table des offres

```
CREATE TABLE IF NOT EXISTS offres (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  source      TEXT NOT NULL CHECK (source IN ('france_travail', 'adzuna')),
  id_source   TEXT,
  hash_offre  TEXT NOT NULL UNIQUE,
  titre       TEXT NOT NULL,
```

```

entreprise      TEXT,
lieu             TEXT,
code_dept       TEXT REFERENCES departements(code_dept),
code_rome       TEXT REFERENCES metiers_rome(code_rome),
categorie       TEXT,
contrat         TEXT CHECK (contrat IN
('CDI','CDD','Intérim','Libéral','Alternance','Autre','Non renseigné')),
temps_travail   TEXT,
experience      TEXT,
salaire_min     REAL CHECK (salaire_min IS NULL OR salaire_min BETWEEN 10000 AND 300000),
salaire_max     REAL CHECK (salaire_max IS NULL OR salaire_max BETWEEN 10000 AND 300000),
salaire_moyen   REAL,
salaire_impute  INTEGER NOT NULL DEFAULT 0 CHECK (salaire_impute IN (0, 1)),
salaire_source  TEXT,
rome_source     TEXT CHECK (rome_source IS NULL OR rome_source IN ('france_travail',
'intitule', 'categorie')),
date_publication TEXT,
description     TEXT,
url            TEXT,
date_import     TEXT NOT NULL
);

```

Contrainte / index	Rôle
hash_offre UNIQUE	Garantit l'absence de doublon, même si le pipeline est relancé (INSERT OR IGNORE)
FOREIGN KEY code_dept, code_rome	Intégrité référentielle, activée par PRAGMA foreign_keys = ON
CHECK source, contrat, salaires, format ROME	Rejette toute valeur hors du domaine autorisé dès l'insertion
6 index	code_dept, code_rome, source, contrat, date_publication, statut : les filtres de l'API

Volumétrie

Table	Lignes
departements	8
metiers_rome	1 100
offres	28 219
indicateurs_tension	1 100

Conformité RGPD

Le registre complet est versionné dans [docs/rgpd.md](#). Points clés :

Rubrique	Mesure
Finalité	Statistiques agrégées sur le marché du travail francilien
Base légale	Intérêt légitime — données publiées par les employeurs, collectées via les API officielles
Personnes concernées	Aucun candidat. Seules les coordonnées de recruteurs peuvent figurer dans le texte libre

Rubrique	Mesure
Minimisation	Description tronquée à 300 caractères ; e-mails et téléphones masqués avant stockage
Conservation	12 mois glissants — <code>purger_offres_anciennes()</code> à chaque exécution (82 offres purgées)
Sécurité	Secrets dans <code>.env</code> ; base ouverte en lecture seule par l'API ; requêtes paramétrées ; clé API
Contrôle	Test automatisé <code>test_aucune_donnee_personnelle</code>

8. C5 — Mettre à disposition les données via une API

L'API REST est développée avec **FastAPI**. Elle expose les offres et les indicateurs de tension en lecture seule, avec une documentation interactive générée automatiquement (OpenAPI / Swagger).

Endpoints

Méthode	Endpoint	Accès	Description
GET	/health	Public	Disponibilité, nombre d'offres et de métiers, date de mise à jour
GET	/offres	Clé	Liste paginée ; filtres departement, code_rome, source, contrat, q (mot-clé)
GET	/offres/{id}	Clé	Détail d'une offre (404 si inexistante)
GET	/metiers	Clé	Indicateurs de tension triés, filtre statut
GET	/metiers/{code_rome}	Clé	Fiche métier : répartition par département et par contrat
GET	/stats/departements	Clé	Volume, densité (offres / 100 km ²), part de CDI, salaire médian, coordonnées
GET	/stats/tension	Clé	Nombre de métiers et d'offres par statut

Sécurité

Risque	Mesure
Accès non autorisé	En-tête X-API-Key obligatoire (sauf /health) → HTTP 403 ; clé lue dans la variable d'environnement PPMT_API_KEY
Injection SQL	Requêtes paramétrées (?) ; aucun paramètre concaténé dans le SQL
Paramètres invalides	Validation Pydantic : motif du département, du code ROME, bornes de pagination → HTTP 422
Extraction massive	limit plafonné à 500 par page
Modification des données	Verbe GET uniquement (405 sinon) ; connexion SQLite en mode lecture seule (mode=ro)
Appels cross-origin	CORS limité au verbe GET

api/main.py – contrôle de la clé

```
_api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def verifier_cle(key: Optional[str] = Security(_api_key_header)) -> None:
    if key != API_KEY:
        raise HTTPException(status_code=403, detail="Clé API invalide ou manquante")
```

api/main.py – endpoint /offres (filtres + pagination, SQL paramétré)

```
@app.get("/offres", response_model=PageOffres, tags=["Offres"],
    dependencies=[Depends(verifier_cle)],
    summary="Liste paginée et filtrable des offres")
def lister_offres(
    departement: Optional[str] = Query(None, pattern=r"^(75|77|78|91|92|93|94|95)$",
    description="Code département"),
```

```

code_rome: Optional[str] = Query(None, pattern=r"^[A-N]\d{4}$", description="Code ROME, ex.
J1502"),
source: Optional[Source] = None,
contrat: Optional[str] = Query(None, description="CDI, CDD, Intérim, Libéral, Autre..."),
q: Optional[str] = Query(None, min_length=2, description="Mot-clé dans l'intitulé"),
limit: int = Query(50, ge=1, le=500),
offset: int = Query(0, ge=0),
):
    filtres, params = [], []
    for col, val in (("code_dept", departement), ("code_rome", code_rome),
                    ("source", source.value if source else None), ("contrat", contrat)):
        if val:
            filtres.append(f"{col} = ?")
            params.append(val)
    if q:
        filtres.append("titre LIKE ?")
        params.append(f"%{q}%")
    where = f"WHERE {' AND '.join(filtres)}" if filtres else ""
    with db() as conn:
        total = conn.execute(f"SELECT COUNT(*) FROM offres {where}", params).fetchone()[0]
        res = lignes(conn, f""SELECT id, source, titre, entreprise, lieu, code_dept, code_rome,
contrat,
                        salaire_moyen, salaire_impute, date_publication, url
FROM offres {where}
ORDER BY date_publication DESC LIMIT ? OFFSET ?""", params +
[limit, offset])
    return {"total": total, "limit": limit, "offset": offset, "resultats": res}

```

Documentation Swagger

API PPMT — Métiers en tension Île-de-France 2.0.0 OAS 3.1

/openapi.json

Offres d'emploi France Travail + Adzuna (8 départements franciliens) et indicateurs de tension par code ROME. Authentification : en-tête **X-API-Key**.

		Authorize
Service ^		
GET	/health	Disponibilité du service (public) v
Offres ^		
GET	/offres	Liste paginée et filtrable des offres
GET	/offres/{offre_id}	Détail d'une offre
Tension ^		
GET	/metiers	Indicateurs de tension par métier (tri décroissant)
GET	/metiers/{code_rome}	Fiche métier : tension, départements, contrats
Statistiques ^		
GET	/stats/departements	Volume d'offres, densité, part de CDI, salaire médian et coordonnées par département
GET	/stats/tension	Répartition des métiers par statut de tension

Figure 6 — Documentation interactive générée par FastAPI (/docs)

Preuve d'exposition des données

Appels réels sur l'API lancée localement (`uvicorn api.main:app`) avec la base produite par le pipeline :

Contrôles d'accès et de validation

```
$ curl http://localhost:8000/health
{"status":"ok","nb_offres":28219,"nb_metiers":1100,"derniere_mise_a_jour":"2026-09-28T16:16:37"}

$ curl http://localhost:8000/metiers # sans clé
{"detail":"Clé API invalide ou manquante"}
HTTP 403

$ curl -H "X-API-Key: $PPMT_API_KEY" "http://localhost:8000/offres?departement=13"
→ HTTP 422 : String should match pattern '^(75|77|78|91|92|93|94|95)$'
```

Fiche métier « Aide ménager / Aide ménagère à domicile » (répartitions tronquées à 3 lignes)

```
$ curl -H "X-API-Key: $PPMT_API_KEY" http://localhost:8000/metiers/K1304
{
  "code_rome": "K1304",
  "libelle": "Aide ménager / Aide ménagère à domicile",
  "nb_offres_ft": 452,
  "nb_offres_adzuna": 17,
  "nb_offres_total": 469,
  "nb_entreprises": 87,
  "part_cdi": 79.7,
  "salaire_median": 22074.0,
  "part_salaire_affiche": 32.0,
  "indice_tension": 1907.2,
  "statut": "TRES EN TENSION",
  "repartition_departements": [
    { "code_dept": "78", "nom": "Yvelines", "nb_offres": 90 },
    { "code_dept": "94", "nom": "Val-de-Marne", "nb_offres": 73 },
    { "code_dept": "77", "nom": "Seine-et-Marne", "nb_offres": 71 }
  ],
  "repartition_contrats": [
    { "contrat": "CDI", "nb_offres": 374 },
    { "contrat": "CDD", "nb_offres": 74 },
    { "contrat": "Non renseigné", "nb_offres": 17 }
  ]
}
```

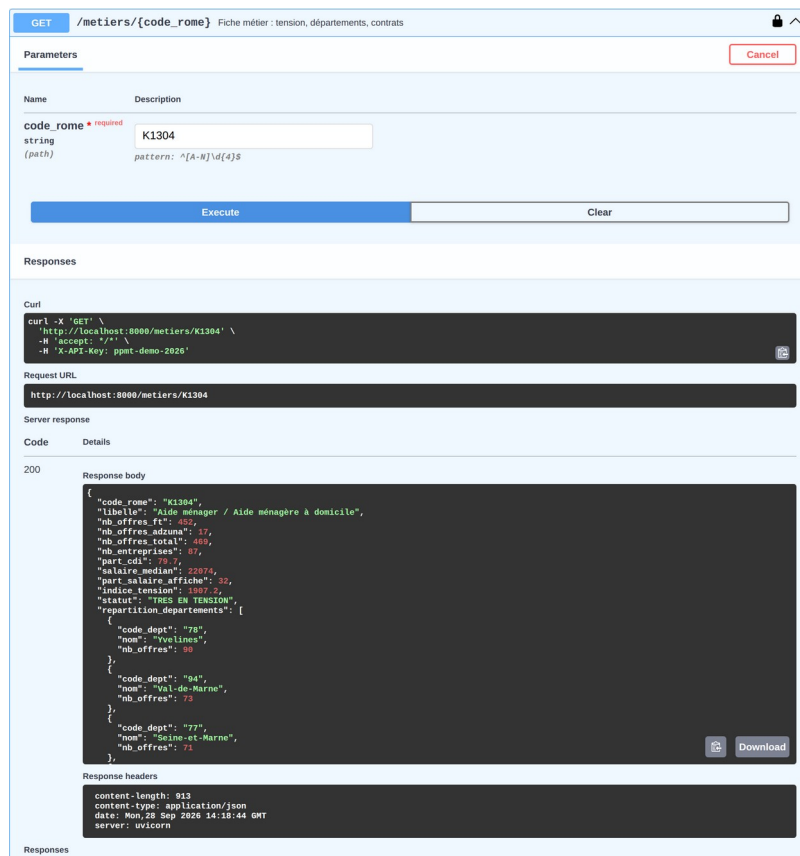


Figure 7 — Même appel depuis Swagger (« Try it out »)

Déploiement

Dockerfile – image légère de l'API

```

# Image de l'API PPMT (C5) – déployable sur Google Cloud Run, Render, Railway...
FROM python:3.11-slim
WORKDIR /app
COPY api/requirements-api.txt .
RUN pip install --no-cache-dir -r requirements-api.txt
COPY api/ ./api/
COPY data/ppmt.db ./data/ppmt.db
ENV PORT=8080 PPMT_DB=/app/data/ppmt.db
# PPMT_API_KEY doit être fournie à l'exécution (secret), ex. : docker run -e PPMT_API_KEY=...
CMD uvicorn api.main:app --host 0.0.0.0 --port ${PORT}

```

L'image n'embarque que FastAPI, Uvicorn et Pydantic (pas pandas) ; elle est déployable telle quelle sur Google Cloud Run ou Render. La clé API est injectée comme secret au lancement.

9. Tests et qualité

`tests/test_pipeline.py` contient **57 tests** exécutés avec `pytest tests/ -v`. Les appels aux API externes sont simulés (objets `FakeSession`) pour tester la pagination et la reprise sur erreur sans dépendre du réseau ni consommer de quota.

Catégorie	Tests	Exemples de vérifications
C1 — collecte (API simulées)	4	arrêt à la dernière page ; plafond 3 150 offres ; reprise après HTTP 429 ; aplatissement du JSON
C2/C3 — règles de préparation	31	conversion des salaires ; cascade de médianes ; codes ROME par intitulé ; référentiel des communes ; libellés de salaire ; départements ; contrats ; pseudonymisation ; priorité France Travail
C4 — base de données	13	aucun salaire vide ; départements géolocalisés ; tables présentes ; intégrité référentielle ; zéro doublon ; périmètre IDF ; aucune donnée personnelle ; CHECK salaires ; ITM moyen = 100
C5 — API	9	/health public ; 403 sans clé ; filtres ; 422 si paramètre invalide ; pagination ; 404 ; 405 sur POST

Extrait de l'exécution – `pytest tests/ -v`

```
test_indicateur_moyen_vaut_100 PASSED [ 82%]
test_statuts_coherents_avec_seuils PASSED [ 84%]
test_health_public PASSED [ 85%]
test_cle_api_obligatoire PASSED [ 87%]
test_stats_departements_avec_coordonnees PASSED [ 89%]
test_offres_filtre_departement PASSED [ 91%]
test_parametre_invalide_422 PASSED [ 92%]
test_pagination PASSED [ 94%]
test_metiers_tries_par_tension PASSED [ 96%]
test_fiche_metier_et_404 PASSED [ 98%]
test_api_lecture_seule PASSED [100%]
===== 57 passed in 3.57s =====
```

10. Service numérique et ouverture Machine Learning

Dashboard Streamlit

Le dashboard (`streamlit run webapp/app.py`) s'adresse aux utilisateurs non techniques. Il propose six onglets avec filtres par département, contrat et secteur :

Onglet	Contenu
Vue d'ensemble	Indicateurs clés, répartition des statuts, top 10 des métiers en tension
Secteurs	Volumes par secteur, contrats France Travail et Adzuna
Compétences & tech	Compétences techniques et comportementales extraites des offres des métiers en tension
Formations & recrutement	Niveaux requis, part de CDI / CDD
Métiers en tension	Statuts, carte des départements dessinée à partir du fichier GeoJSON
Salaires	Comparaison France Travail / Adzuna par métier, secteur et département

Expérimentation ML : une fuite de données identifiée

En V1, un Random Forest prédisait l'indice de tension avec un R^2 de 0,995. En analysant le calcul, j'ai constaté que l'indice est lui-même calculé à partir des variables d'entrée ($\text{nb_offres_total} \div \text{moyenne} \times 100$) : le modèle retrouvait une formule, il ne prédisait rien. Le script actuel reproduit ce diagnostic ($R^2 = 1,00$ en régression linéaire), puis le corrige.

V2 : classification « en tension » (indice > 100) à partir du **profil** des offres uniquement — part de CDI, CDD et intérim, salaire, part de salaires non affichés, expérience demandée, longueur des descriptions, famille ROME — sans aucune variable de volume. Les 446 métiers ayant au moins 10 offres sont évalués en validation croisée stratifiée à 5 plis. Trois modèles sont comparés ; le modèle retenu est celui qui obtient la meilleure AUC. Pour repère, un classement au hasard obtient 0,49.

Modèle	ROC-AUC	F1	Exactitude équilibrée
Régression logistique	0,58	0,53	0,55
Random Forest	0,82	0,74	0,73
XGBoost	0,84	0,77	0,75

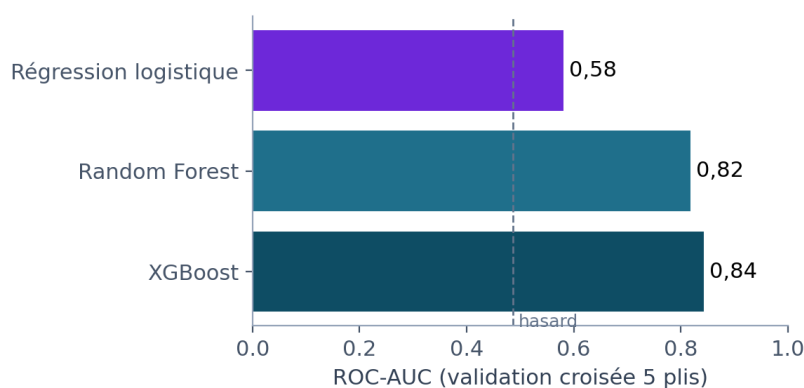


Figure 8 — Comparaison des modèles (pointillés : classement au hasard)

XGBoost est retenu (AUC 0,84, F1 0,77) : le profil des offres contient un signal réel. Les variables les plus explicatives sont la longueur des descriptions, la famille de métiers (commerce, logistique, services à la

personne), la part de temps partiel, d'intérim et de CDD : les métiers en tension recrutent davantage sur des contrats courts et publient des annonces plus courtes. Cette partie ne relève pas du bloc BC01 ; elle ouvre sur le bloc suivant.

11. Bilan et perspectives

Correspondance avec le référentiel

Compétence	Livrables	Preuves
C1 — Collecte automatisée	<code>src/collect.py</code>	2 API (OAuth2, clé) + 2 fichiers de référence (contours, communes) réutilisés en base, dans l'API et sur la carte ; pagination ; reprise ; journaux ; 4 tests
C2 — Préparation	<code>src/prepare.py</code>	10 problèmes traités (méthodes V1 et V2 comparées), cascade de médianes pour les salaires ; schéma commun ; RGPD ; tests unitaires
C3 — Agrégation	<code>SQL_INDICATEURS</code> (store.py)	8 indicateurs par métier : SQL (GROUP BY, fonction fenêtre) + salaire médian
C4 — Base de données	<code>src/store.py</code> , <code>data/ppmt.db</code> , <code>docs/rgpd.md</code>	MPD 4 tables, FK, CHECK, UNIQUE, index ; registre RGPD ; purge
C5 — API	<code>api/main.py</code> , <code>Dockerfile</code>	7 endpoints, clé API, Swagger, validation, tests 403/404/405/422

Chiffres clés

Indicateur	Valeur
Offres en base	28 219 (France Travail 23 691 · Adzuna 4 528)
Métiers ROME couverts	1 100
Métiers « très en tension »	185
Départements	8
Tests automatisés	57 (100 % réussis)
Durée d'exécution du pipeline (hors collecte)	≈ 5 secondes

Limites

- L'indice de tension ne mesure que la demande des employeurs ; il manque le nombre de demandeurs d'emploi par métier.
- La correspondance catégorie Adzuna → ROME est grossière (19 codes) : elle surreprésente certains métiers (ex. développeur, M1805).
- Une seule collecte (mai 2026) : pas encore de série temporelle.

Perspectives

- **Offre / demande** : intégrer les demandeurs d'emploi par code ROME (statistiques France Travail / DARES) pour calculer un vrai ratio de tension.
- **Historisation** : conserver un instantané quotidien des indicateurs pour prévoir la tension à 3 mois (modèles de séries temporelles).

- **Déploiement** : publier l'API sur Google Cloud Run et le dashboard sur Streamlit Cloud ; orchestration par Airflow.
- **Passage à l'échelle** : extension aux autres régions, migration vers PostgreSQL.

Annexes

A — Lancement rapide

```
git clone https://github.com/Clairelucie/PPMT.git && cd PPMT
pip install -r requirements.txt
cp .env.example .env                # renseigner les identifiants France Travail et Adzuna
python run_pipeline.py --collect     # C1 → C4 (sans --collect : repart des CSV versionnés)
pytest tests/ -v                    # 57 tests
uvicorn api.main:app --reload        # API → http://localhost:8000/docs
streamlit run webapp/app.py         # dashboard → http://localhost:8501
```

B — Dépendances

```
# PPMT – dépendances (RNCP37827BC01 – Artefact 2026)
# Collecte & préparation (C1 → C3)
pandas>=2.0
numpy>=1.23
requests>=2.28
python-dotenv>=0.21
# API (C5)
fastapi>=0.110
uvicorn[standard]>=0.29
pydantic>=2.5
# Tests
pytest>=8.0
httpx>=0.27
# Machine Learning (expérimentation)
scikit-learn>=1.3
xgboost>=1.7
joblib>=1.2
# Dashboard
streamlit>=1.30
plotly>=5.10
```

C — Clé API de démonstration

Clé utilisée pour la démonstration devant le jury : **ppmt-demo-2026**. En production, la clé est fournie par la variable d'environnement **PPMT_API_KEY** et n'apparaît pas dans le code.

D — Glossaire

Terme	Définition
ROME	Répertoire Opérationnel des Métiers et des Emplois (France Travail) — un code par métier, ex. J1502
ITM	Indice de tension PPMT : volume d'offres d'un métier rapporté au métier moyen (× 100)
BMO	Enquête Besoins en Main-d'Œuvre de France Travail
DARES	Direction de l'animation de la recherche, des études et des statistiques (ministère du Travail)
OAuth2	Protocole d'autorisation ; ici, flux « client credentials » entre applications
Fuite de données	Information sur la cible présente dans les variables d'entrée, qui rend un score artificiellement élevé

Terme	Définition
RGPD	Règlement général sur la protection des données

Dossier rédigé en octobre 2026 — Compétences RNCP37827BC01 (C1 à C5)